Software Requirements Specification (SRS) for *Kiko*

TophUwO

December 2023

Contents

Version History **2**

1 Introduction **3**
1.1 Purpose of This Document . . . 3
1.2 Document Conventions . . . 3
1.3 Background and Rationale . . . 3
1.4 Product Feature Overview . . . 3
1.5 Stakeholders and Audience . . . 3
1.6 Glossary . . . 4
1.7 External References . . . 4

2 General Description **5**
2.1 Perspective . . . 5
2.2 Product Features . . . 5
2.3 Application Users . . . 5
2.4 User Classes and Characteristics . . . 5
2.5 Operating Environment . . . 5
2.6 Developer Limitations . . . 5
2.7 User Documentation . . . 6
2.8 Assumptions . . . 6
2.9 Distribution . . . 6
2.10 Optional Features . . . 6

3 Core System Features **7**
3.1 Creation and Population of Documents and Charts . . . 7
3.2 Document Rendering . . . 7
3.3 Document, Chart, and Model Customization . . . 7

4 External Interface Requirements **8**
4.1 User Interfaces Overview . . . 8
4.2 Hardware and Software Interfaces . . . 8

5 Specific Requirements **9**
5.1 Functional Requirements . . . 9
5.1.1 Creation and Population of Documents and Charts . . . 9
5.1.2 Document Rendering . . . 9
5.1.3 Document, Chart, and Model Customization . . . 9
5.2 Non-functional Requirements . . . 9
5.2.1 Performance . . . 9
5.2.2 Thread-Safety . . . 9
5.2.3 Standard Conformance . . . 9
5.2.4 Internationalization . . . 9
5.2.5 Software Quality Requirements . . . 9

A Supplementary Resources **11**

B OOA Diagrams **12**
B.1 OOA package diagram . . . 12
B.2 Use-Case Diagrams . . . 12
B.3 OOA Class Diagram . . . 12

Version History

Below is a chronological list of changes, alongside metadata. Version *1.0* marks the first release version of this document. Dates used in this table are of the format ***MM/DD/YYYY***.

Date	Version	Contributor(s)	Changes
12/20/2023	0.1	TophUwO	**initial commit**; setup document structure
12/20/2023	0.2	TophUwO	**ADD** initial contents for section *Introduction*; improve document structure
12/20/2023	0.2.5	TophUwO	**ADD** contents for all sub-sections of *Non-functional Requirements*
12/24/2023	0.4	TophUwO	**ADD** contents for section *Optional Features*, *Standard Conformance*, and root section *External Interface Requirements*
12/24/2023	0.4.1	TophUwO	**ADD** contents (table) for section *Product Features*
12/27/2023	0.5	TophUwO	**ADD** contents for remaining sub-sections in section *General Description*; **ADD** supplementary image for CSS box model
12/29/2023	0.6	TophUwO	**ADD** contents for section *Core System Features*

1 Introduction

This chapter serves as a general introduction to the product, its features, and its working environment. The features and requirements mentioned here will be discussed and elaborated on in chapter 3 and 4.

1.1 Purpose of This Document

This document describes the software requirements specification (SRS) for a *CSS*-based chart generator, from now on throughout the rest of this document referred to as *Kiko*, taking model data and rendering this data as a standard *SVG* document, ready for rasterization and/or embedding. This document was created in accordance with the *IEEE 830* standard, is part of the internal developer documentation, and serves as a benchmark of what the application is intended to do and be used for.

1.2 Document Conventions

In this document, technical and other domain-specific terms relevant to the implementation are in *italics* while table heads are rendered in **boldface**. See section *Glossary* for an explanation of important technical terms used in the context of the product's domain. Other nomenclature (also in *italics*) may be further explained by either footnotes or in the section *External References*. Many terms in *italics* are also hyperlinks to other sections within this document.
This document manages its own version history (see section *Version History*). Each commit holds data such as *Date*, *Version*, *Contributor(s)*, and *Changes*. Changes begin with a verb in infinitive form, all-caps, in boldface, describing the change's category, e.g. **ADD**, **REMOVE**, **FIX**, etc.
Figures such as GUI mock-ups and OOA diagrams can be found in the Appendix A and B, respectively.
The language used for this document and all external resources is *English (United States)*.

1.3 Background and Rationale

Desktop applications are going out of fashion. With the advent of the *World Wide Web*, mankind became exposed to a host of new technologies and platforms which would allow the creation of interactive software and software components. Today, web applications are ubiquitous. From simple scribble applications to complex enterprise management software, the platform *Web Browser* is ever-growing. Many such software deals with data, and when data is dealt with, it is often visualized in the form of diagrams and charts. Traditionally, diagrams were created either manually or semi-automatically by data visualization software and spreadsheet applications. The output formats were raster-based in most cases, leaving little possibility for view-dependent optimization, meaning it was sheer impossible to provide good visual representation for all possible device types and display dimensions. This is where *Kiko* comes into play. Kiko is a data-visualization API, generating SVG documents from given data models, producing beautiful charts with a variety of customization options. SVG has the advantage that it can be scaled arbitrarily which allows the user to display razor-sharp data visualizations for all possible device types. Furthermore, if wished, the resulting SVG document can be rasterized by any application suitable for the job. I believe the creation of this software will enhance the workflow of data visualization experts across the globe.

1.4 Product Feature Overview

At its core, Kiko implements a CSS-based chart generator, producing SVG documents. This chart generator will be exposed in the form of an application programming interface (API), allowing it to be easily embedded into existing or new code-bases. The central data structure is a *document* which hold at least one *chart* which holds one or more *models*. A document can contain multiple aligned charts. To enhance the visual representation, the generator supports a host of CSS-like style options for each element.

1.5 Stakeholders and Audience

Since this is a personal project, the only stakeholder is the project manager, developer and maintainer, TophUwO. This document is also meant for future contributors of the software described within this document.

1.6 Glossary

Below is a curated list of context-specific nomenclature used throughout this document, alongside brief explanations. Please note that the explanations given are supposed to serve as a very broad introductory phrase only. Additional research has to be carried out in order to gather information necessary to satisfy the requirements outlined by this document.

Term	**Explanation**
SVG	*Scalable Vector Graphics*; XML-based format where graphical elements are represented as mathematical expressions rather than pixels
CSS	*Cascading Style Sheets*; formal definition of styling rules for document elements, commonly used with HTML

1.7 External References

Below is a non-exhaustive list of useful resources that are likely to be frequently consulted throughout the conceptualization and implementation of the software described by this document. The resources presented beneath may serve as a starting point but do in no way represent a complete and necessarily sufficient resource for the entirety of the subject. Additional research is expressly endorsed.

Resource	**URL**
IEEE 830 specification	`http://www.math.uaa.alaska.edu/~afkjm/cs401/IEEE830.pdf`
DOXYGEN	`https://www.doxygen.nl/`
ISO/IEC 25010:2011	`https://iso25000.com/index.php/en/iso-25000-standards/iso-25010`
CSS 2.1	`https://www.w3.org/TR/CSS21/`
SVG 1.1 (2nd revision)	`https://www.w3.org/TR/SVG11/`

2 General Description

2.1 Perspective

The software described by this document is an embeddable software, generating SVG documents, and is supposed to be used by myself for other personal projects. A public release is planned, and will be issued once the software is mature enough.

2.2 Product Features

Below is a basic (read: *high-level*) overview of the API's features. Almost all functional requirements listed here have sub-requirements and will be split-up in more finely-distinguished use-cases.

Feature Title	**Description**
initialize document	This must be done before initializing *charts* and *models*. There can be multiple documents initialized at the same time.
initialize chart	Initializes a new chart of the current document; allowing the user to set global options, style-sheets, and axis information.
initialize model	Initializes data-model so that data can be added to it.
submit model data	Add data to the model so it will be included in the generated SVG document.
update model data	Replace existing data with new data without re-initializing the model or other parent data structures.
delete node	Remove chart, model, or model data from the document.
update document and model settings	Change global (document) and model-global settings.
generate SVG document	Produce ready-to-use SVG document using the current document or chart. Optionally, the output can be directly written to a file.

2.3 Application Users

This software is to be used by data scientists, analysts, framework developers. It can be assumed that they are familiar with the C programming language as well as basic HTML and CSS. To aid in data submission, knowledge of the *Qt Charts* framework is recommended.

2.4 User Classes and Characteristics

The software is not user-based. Hence, there is no concept of a user or a privilege. Some operations provided by the library may involve system calls for I/O which may require elevated privileges for the calling application instance.

2.5 Operating Environment

The software should be able to run on any device that has a working C11 compiler and sufficient memory. All OS-specific functionality is to be wrapped by the library if the required level of control exceeds that level provided by a standard library implementation. Thus, the software does not directly interact with hardware.

2.6 Developer Limitations

There are little limitations in place. The application should be written in standard C11. Third-party frameworks and libraries may be used if they are not licensed under a strong copyleft license like *GPL*. The language used in the codebase and the developer documentation must be *English (United States)*. There is no limit given regarding the memory footprint, but it should be kept to a minimum where possible (i.e. no or very little preemptive allocations, minimize dynamic data structures and indirection as much as possible).

2.7 User Documentation

Aside from the SRS provided by this document, there will be a *DOXYGEN*-based online developer documentation available. A printable PDF edition is not planned.

2.8 Assumptions

The software should be cross-platform, using only platform-independent interfaces to use the functionality exposed by the host OS (except for when wrappers are implemented that abstract native functionality).

2.9 Distribution

The software's main distribution format is compiled, ready to be used with any suitable codebase. The software's source code is not to be distributed alongside the compiled binaries. Binaries must be provided for the most common C-compilers (GCC, Clang, MSVC, ICC). A release of the software's full source code under a non-copyleft open-source license is thinkable. Details regarding this must be worked out in time.

2.10 Optional Features

The main optional feature is supporting more diagram types outside of the required ones. Optional data models to be implemented are:

- radar chart
- spline chart
- histogram (normal/radial)
- candlestick chart
- step-line chart
- bump chart
- waffle chart
- matrix chart

Also, required data models can be extended in the following way: A *bar chart* can be displayed as a *radial bar chart*. *Pie charts* can be displayed as *exploded*, *partly-exploded* (i.e. only certain segments offset), or as a *strip* (i.e. like a single bar in a stacked bar chart).
Furthermore, there could be more style options like CSS-like style-sheets (subset of *CSS 2.1*), more document generation options (like embedding style-sheets, more advanced SVG features like animation), support for JS scripting, etc.

3 Core System Features

3.1 Creation and Population of Documents and Charts

It should be possible to initialize *documents*, *charts*, and *models*. A *document* is the parent data structure of all of Kiko's objects. Documents cannot be nested. A document can hold an arbitrary number of *charts* which can be aligned using a global *layout* specified by the *document*. Available layouts should be *vertical* (align charts in a column), *horizontal* (align charts in a row), and *grid* (align charts in an MxN grid).
Charts are the main visualization components of *models*. After initializing a *chart*, *axis* can be initialized inside the *chart*. Each axis can have an alignment (i.e. *left*, *top*, *center*, *right*, and *bottom*) and an axis type (i.e. *horizontal* and *vertical*). There can be an arbitrary number of axis for each alignment position. Axis can have *min-max* ranges or have a collection of definitive labels attached.
Models are containers for actual data-points which have to be given in a specific way depending on the model type. The required data models are:

- bar chart (horizontal/vertical, stacked/unstacked, grouped/ungrouped)
- heat map
- bubble chart
- scatter plot (with/without regression with polynomial function)
- pie chart (donut, half-donut)
- area chart (normal/stacked)
- line chart

3.2 Document Rendering

At any point in time, a *document* or any *chart* can be rendered to an SVG document by using a special API call. During this call, the object is immutable. The document can be written to a string buffer and also a file.

3.3 Document, Chart, and Model Customization

One central feature of the API is elaborate customization options. The default way of specifying style information is through a special API call. There should be a default style built-in that every element is initialized with. When the SVG document is generated, the library should use the specified styles to produce the corresponding output. There should be a way to generate a CSS-like document from all style options specified for the given element.

4 External Interface Requirements

4.1 User Interfaces Overview

The software's user interface is the only interface it provides, a C-based software API. The API will be structured to resemble an object-oriented approach. By default, there will be no way to access the API from a graphical user interface.

4.2 Hardware and Software Interfaces

The software does not directly interact with any hardware or software. Thus, no drivers are to be shipped alongside the API. OS-specific features are either wrapped by the software itself or provided by the C Standard Library.

5 Specific Requirements

5.1 Functional Requirements

5.1.1 Creation and Population of Documents and Charts

5.1.2 Document Rendering

5.1.3 Document, Chart, and Model Customization

5.2 Non-functional Requirements

5.2.1 Performance

Performance is of importance for all API calls. No operation should introduce any significant overhead when used with realistic inputs. Data structures should be optimized for easy and efficient traversion (e.g. for chart generation) and easy look-up (e.g. for style-sheets). Where possible, contiguous data structures should be used. Furthermore, the memory foot-print should be low but not at the cost of runtime performance.

5.2.2 Thread-Safety

All API calls should be thread-safe at the process level. Hence, inter-process communication is not a required feature. This behavior may be turned on or off via a compiler switch, i.e. there should be a way to compile the software without thread-safety features enabled in case this feature is not needed. Additionally, there should be no timeouts for waiting operations. Document generation should be done on a single core in a fully thread-safe operation.

5.2.3 Standard Conformance

The API should be written in pure C11, with no extensions enabled. It must be compilable at the highest warning/pedantry level the compiler provides. What's more, to make it as simple as possible to implement/find a rasterizer that can render the generated SVG documents to a bitmap format, the library is restricted to SVG 1.1-only features; animation and scripting are optional features. More advanced features introduced in other versions of the SVG standard are usable, but have to be enabled specifically via either a compiler flag or a run-time setting.

5.2.4 Internationalization

No special care has to be taken to internationalize the API. The API's calls should be in *English (United States)* as well as the contained error messages. Translating the error messages is an entirely optional feature of very low priority.

5.2.5 Software Quality Requirements

Below is a list of all *ISO/IEC 25010:2011* software quality requirements alongside an integer value, the level of importance, in the interval [1, 5] where *5* denotes very high importance and *1* denotes low importance relative to level *5*.

Requirement Class	Description	Priority
functional suitability	This characteristic represents the degree to which a product or system provides functions that meet stated and implied needs when used under specified conditions.	**4**
performance efficiency	This characteristic represents the performance relative to the amount of resources used under stated conditions.	**3**
compatibility	Degree to which a product, system or component can exchange information with other products, systems or components, and/or perform its required functions while sharing the same hardware or software environment.	**5**

usability	Degree to which a product or system can be used by specified users to achieve specified goals with effectiveness, efficiency and satisfaction in a specified context of use.	**3**
reliability	Degree to which a system, product or component performs specified functions under specified conditions for a specified period of time.	**3**
security	Degree to which a product or system protects information and data so that persons or other products or systems have the degree of data access appropriate to their types and levels of authorization.	**1**
maintainability	This characteristic represents the degree of effectiveness and efficiency with which a product or system can be modified to improve it, correct it or adapt it to changes in environment, and in requirements.	**2**
portability	Degree of effectiveness and efficiency with which a system, product or component can be transferred from one hardware, software or other operational or usage environment to another.	**4**

A Supplementary Resources

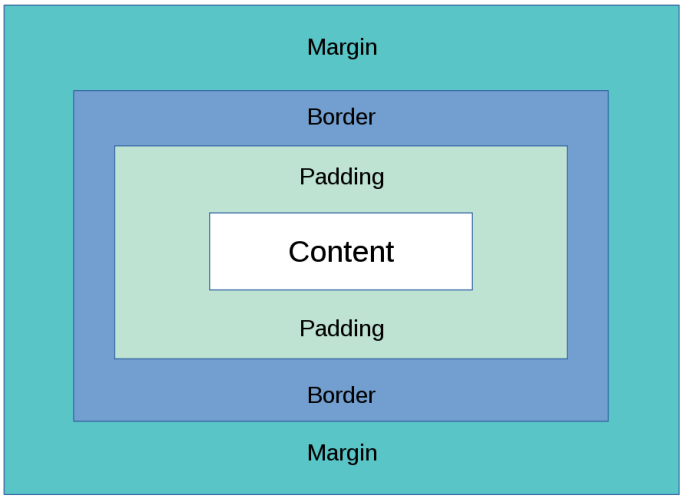

Figure 1: CSS box model (Source: *Wikimedia*)

B OOA Diagrams

B.1 OOA package diagram

B.2 Use-Case Diagrams

B.3 OOA Class Diagram